



INFORME DEL TRABAJO

I. PORTADA

Tema:	Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Silva Camuendo Luis Alexander
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DEL TRABAJO

2.1 Objetivos

General:

Implementar y analizar Tipos de Datos Abstractos (TDA) basados en listas simplemente enlazadas circulares en Java, aplicando operaciones de inserción, eliminación y recorrido cíclico para resolver problemas de gestión y simulación.

Específicos:

- Diseñar interfaces y clases de implementación que separen el contrato del TDA de la estructura interna de nodo.
- Aplicar las propiedades de las listas circulares en operaciones de inserción, eliminación, recorrido continuo y conteo de elementos.
- Implementar aplicaciones de listas circulares en la simulación Round-Robin, el problema de Josephus y una playlist musical.
- Verificar el funcionamiento de los cinco programas mediante pruebas de escritorio, resultados de ejecución y menús de entrada por teclado.

2.2 Ejercicios

Ejercicio 1. Lista circular básica

1. Ejercicio

Implemente en Java una lista simplemente enlazada circular que permita insertar al inicio, insertar al final, mostrar todos los elementos, verificar si la lista está vacía y contar el número de elementos. Requisito adicional: explique con un ejemplo por qué en una lista circular el último nodo debe apuntar al primero.

2. Solución propuesta

La solución separa el contrato abstracto, la implementación de la estructura circular y la interacción con el usuario. Las responsabilidades se distribuyen de la siguiente manera:

Clase / módulo	Responsabilidad
ListaCircularTDA	Interfaz que define insertar al inicio/final, mostrar, consultar si está vacía y contar.
ListaCircularBasicaImpl	Implementa la lista circular mediante nodos enlazados, la referencia ultimo y el contador tamaño.
Nodo	Clase interna que almacena dato y siguiente.
PruebaListaCircularBasica	Contiene el menú por teclado y utiliza el TDA mediante la interfaz.



3. ¿Qué es un TDA y dónde está implementado?

Un Tipo de Dato Abstracto define las operaciones disponibles y las reglas del dato sin obligar al usuario a conocer la representación interna. En este ejercicio, el contrato está en la interfaz y la implementación concreta se encuentra en la clase indicada a continuación:

```
public interface ListaCircularTDA {  
    void insertarInicio(int valor);  
    void insertarFinal(int valor);  
    void mostrar();  
    boolean estaVacia();  
    int contar();  
}
```

La implementación real del contrato corresponde a `ListaCircularBasicaImpl` implements `ListaCircularTDA`. El programa principal trabaja con una referencia del tipo de la interfaz, lo que mantiene separado el uso del TDA de sus detalles internos.

4. Relación entre requisitos y código

Requisito del enunciado	Aplicación en el código Java
Insertar al inicio	<code>insertarInicio(int valor)</code>
Insertar al final	<code>insertarFinal(int valor)</code>
Mostrar	<code>mostrar()</code>
Lista vacía	<code>estaVacia()</code>
Contar	<code>contar()</code>

5. Bibliotecas y entrada/salida

```
import java.util.Scanner;
```

La clase de prueba utiliza `Scanner` para capturar las opciones y datos ingresados por teclado. La salida se realiza con `System.out.print` y `System.out.println`. La estructura circular no necesita colecciones de `java.util` porque los enlaces se gestionan directamente mediante nodos.

6. Estructura principal del TDA

La representación interna se encuentra encapsulada en la clase de implementación. Sus campos permiten conservar el estado del TDA sin exponer directamente los nodos al programa cliente.

```
private static class Nodo {  
    int dato;  
    Nodo siguiente;  
    Nodo(int dato) { this.dato = dato; }  
}  
  
private Nodo ultimo;  
private int tamano;
```



7. Implementación de la interfaz

La clase concreta declara ListaCircularBasicaImpl implements ListaCircularTDA. Los métodos definidos por la interfaz se implementan con @Override, de modo que el programa cliente puede invocarlos a través del tipo abstracto sin depender de la clase concreta.

```
public class ListaCircularBasicaImpl implements ListaCircularTDA {  
    // métodos @Override del TDA  
}
```

8. Nodo o elemento de la estructura circular

La clase interna Nodo es un detalle de implementación. Cada nodo contiene el dato del ejercicio y la referencia siguiente. La circularidad se obtiene evitando que el último enlace sea null y conectándolo con el primer nodo.

9. Construcción e instanciación del TDA

```
ListaCircularTDA lista = new ListaCircularBasicaImpl();
```

La instanciación crea la implementación concreta, pero la variable del cliente se declara con el tipo de la interfaz. Esto permite utilizar el contrato del TDA y mantener ocultos los detalles de los nodos.

10. Operaciones principales

Operación	Resultado / descripción
insertarInicio(int valor)	Crea un nodo y lo coloca como primer elemento; si la lista estaba vacía, el nodo se apunta a sí mismo.
insertarFinal(int valor)	Inserta después del último, enlaza el nuevo con el primero y actualiza ultimo.
mostrar()	Recorre con do-while desde ultimo.siguiete hasta regresar al primero.
estaVacia()	Devuelve ultimo == null.
contar()	Devuelve tamano.

11. Funcionamiento interno de las operaciones

```
ListaCircularTDA lista = new ListaCircularBasicaImpl();  
Scanner sc = new Scanner(System.in);  
...  
switch (opcion) {  
    case 1: lista.insertarInicio(leerEntero(sc)); break;  
    case 2: lista.insertarFinal(leerEntero(sc)); break;  
    case 3: lista.mostrar(); break;  
    case 4: System.out.println(lista.estaVacia()); break;  
    case 5: System.out.println(lista.contar()); break;  
}
```



El punto clave es ultimo.siguiete: cuando la lista no está vacía representa siempre al primer nodo. Insertar al inicio cambia ese enlace; insertar al final también actualiza ultimo.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / uso
dato, tamano, opcion	int	Primitivo	Valores enteros y cantidad de nodos.
ultimo, siguiente	Nodo	Referencia	Enlaces que forman el ciclo.
lista	ListaCircularTDA	Referencia	Acceso al TDA mediante su interfaz.
sc	Scanner	Referencia	Lectura de datos desde teclado.

13. Diagrama de clases

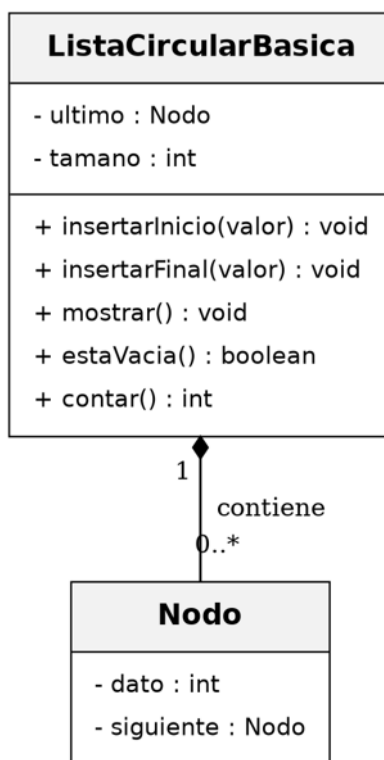


Figura 1. Diagrama de clases de ListaCircularBasica y Nodo.

14. Especificación del TDA

Estado e invariantes de representación:

- Lista vacía: ultimo == null y tamano == 0.
- Lista no vacía: ultimo.siguiete es el primer nodo.
- Ningún nodo activo tiene siguiente == null.
- Después de una vuelta completa se regresa al primer nodo.
- tamaño coincide con el número real de nodos.



Las operaciones públicas válidas son las definidas en la interfaz mostrada en el paso 3. Mientras se respeten los invariantes anteriores, la estructura conserva su comportamiento circular después de cada operación.

15. Diagrama de flujo y menú de ejecución

Flujo general del programa:

INICIO → crear ListaCircularBasicaImpl → mostrar menú → leer opción → ejecutar insertar/mostrar/consultar → volver al menú → opción 0 → FIN.

Menú real de ejecución por teclado:

1. Insertar al inicio
2. Insertar al final
3. Mostrar todos los elementos
4. Verificar si la lista está vacía
5. Contar el número de elementos
0. Salir

Entrada por teclado	Acción	Resultado esperado / verificado
4	Consultar estado inicial	true
1, 10	Insertar al inicio	10
2, 20	Insertar al final	10 -> 20
1, 5	Insertar al inicio	5 -> 10 -> 20
3	Mostrar	Vuelve al primero: 5
5	Contar	3
0	Salir	Saliendo...

16. Prueba de escritorio, resultados verificados y análisis solicitado

La siguiente prueba resume el comportamiento comprobado del TDA con una secuencia representativa de operaciones:

Paso	Operación	Resultado verificado
1	Consultar vacía	true
2	InsertarInicio(10)	10
3	InsertarFinal(20)	10 -> 20
4	InsertarInicio(5)	5 -> 10 -> 20
5	mostrar()	5 -> 10 -> 20 -> (vuelve al primero: 5)
6	contar()	3

Pregunta textual del enunciado: Explique con un ejemplo por qué en una lista circular el último nodo debe apuntar al primero.

Respuesta: Ese enlace cierra el ciclo. Por ejemplo, en 5 -> 10 -> 20, el nodo 20 debe apuntar nuevamente a 5. Así, después de recorrer el último elemento se continúa desde el primero sin encontrar null.



Ejercicio 2. Inserción y eliminación controlada

1. Ejercicio

Desarrolle un programa en Java que implemente una lista circular y permita insertar un elemento en una posición específica, eliminar un elemento por posición, eliminar un elemento por valor e imprimir la lista antes y después de cada operación. Analice qué ocurre con una lista vacía, una lista con un solo nodo y una lista con varios nodos.

2. Solución propuesta

La solución separa el contrato abstracto, la implementación de la estructura circular y la interacción con el usuario. Las responsabilidades se distribuyen de la siguiente manera:

Clase / módulo	Responsabilidad
ListaCircularControladaTDA	Define inserción por posición, eliminación por posición/valor y consultas.
ListaCircularControladaImpl	Implementa validaciones, recorridos y reconexión de nodos.
Nodo	Almacena dato y siguiente.
PruebaListaCircularControlada	Presenta el menú e imprime el estado antes y después.

3. ¿Qué es un TDA y dónde está implementado?

Un Tipo de Dato Abstracto define las operaciones disponibles y las reglas del dato sin obligar al usuario a conocer la representación interna. En este ejercicio, el contrato está en la interfaz y la implementación concreta se encuentra en la clase indicada a continuación:

```
public interface ListaCircularControladaTDA {  
    void insertarEnPosicion(int valor, int posicion);  
    boolean eliminarPorPosicion(int posicion);  
    boolean eliminarPorValor(int valor);  
    void mostrar();  
    boolean estaVacía();  
    int contar();  
}
```

La implementación real del contrato corresponde a ListaCircularControladaImpl implements ListaCircularControladaTDA. El programa principal trabaja con una referencia del tipo de la interfaz, lo que mantiene separado el uso del TDA de sus detalles internos.

4. Relación entre requisitos y código

Requisito del enunciado	Aplicación en el código Java
Insertar por posición	insertarEnPosicion(int,int)
Eliminar por posición	eliminarPorPosicion(int)
Eliminar por valor	eliminarPorValor(int)
Antes/después	PruebaListaCircularControlada llama mostrar() antes y después



5. Bibliotecas y entrada/salida

```
import java.util.Scanner;
```

La clase de prueba utiliza Scanner para capturar las opciones y datos ingresados por teclado. La salida se realiza con System.out.print y System.out.println. La estructura circular no necesita colecciones de java.util porque los enlaces se gestionan directamente mediante nodos.

6. Estructura principal del TDA

La representación interna se encuentra encapsulada en la clase de implementación. Sus campos permiten conservar el estado del TDA sin exponer directamente los nodos al programa cliente.

```
private static class Nodo {  
    int dato;  
    Nodo siguiente;  
    Nodo(int dato) { this.dato = dato; }  
}  
  
private Nodo ultimo;  
private int tamano;
```

7. Implementación de la interfaz

La clase concreta declara ListaCircularControladaImpl implements ListaCircularControladaTDA. Los métodos definidos por la interfaz se implementan con @Override, de modo que el programa cliente puede invocarlos a través del tipo abstracto sin depender de la clase concreta.

```
public class ListaCircularControladaImpl implements ListaCircularControladaTDA {  
    // métodos @Override del TDA  
}
```

8. Nodo o elemento de la estructura circular

La clase interna Nodo es un detalle de implementación. Cada nodo contiene el dato del ejercicio y la referencia siguiente. La circularidad se obtiene evitando que el último enlace sea null y conectándolo con el primer nodo.

9. Construcción e instanciación del TDA

```
ListaCircularControladaTDA lista =  
    new ListaCircularControladaImpl();
```

La instanciación crea la implementación concreta, pero la variable del cliente se declara con el tipo de la interfaz. Esto permite utilizar el contrato del TDA y mantener ocultos los detalles de los nodos.

10. Operaciones principales



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Operación	Resultado / descripción
insertarEnPosicion(v,p)	Acepta $0 \leq p \leq \text{tamano}$; reutiliza métodos internos para inicio/final y enlaza en posiciones intermedias.
eliminarPorPosicion(p)	Valida $0 \leq p < \text{tamano}$ y contempla lista de un nodo, primero, intermedio y último.
eliminarPorValor(v)	Recorre como máximo tamano nodos y elimina la primera coincidencia.
mostrar()	Imprime una sola vuelta del círculo.
contar()/estaVacia()	Consultan cantidad y estado sin modificar la lista.

11. Funcionamiento interno de las operaciones

```
System.out.print("Antes: ");  
lista.mostrar();  
lista.insertarEnPosicion(valor, posIns);  
System.out.print("Después: ");  
lista.mostrar();  
  
lista.eliminarPorPosicion(posDel);  
lista.eliminarPorValor(valDel);
```

Las operaciones validan posiciones antes de recorrer. Para eliminar, el enlace del nodo anterior se redirige al siguiente del nodo objetivo; si el objetivo era el último, ultimo se actualiza.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / uso
dato, tamano, posicion	int	Primitivo	Datos, tamaño e índices.
ultimo, siguiente	Nodo	Referencia	Mantienen la lista circular.
lista	ListaCircularControladaTDA	Referencia	Contrato del TDA.
opcion	int	Primitivo	Control del menú.



13. Diagrama de clases

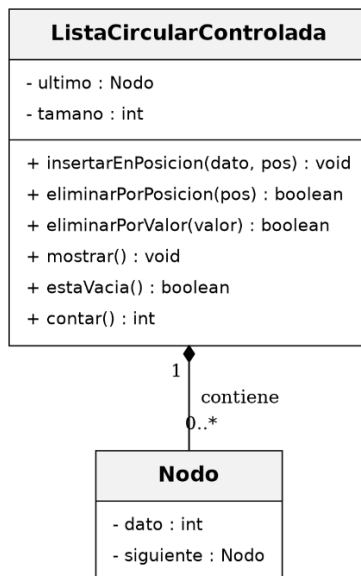


Figura 2. Diagrama de clases de ListaCircularControlada y Nodo.

14. Especificación del TDA

Estado e invariantes de representación:

- Lista vacía: `ultimo == null` y `tamaño == 0`.
- Si no está vacía, `ultimo.siguiente` es el primero.
- Índices de elementos válidos: `0..tamaño-1`.
- Para insertar también se admite `posicion == tamaño`.
- Después de eliminar, `tamaño` disminuye exactamente en uno y el ciclo permanece cerrado.

Las operaciones públicas válidas son las definidas en la interfaz mostrada en el paso 3. Mientras se respeten los invariantes anteriores, la estructura conserva su comportamiento circular después de cada operación.

15. Diagrama de flujo y menú de ejecución

Flujo general del programa:

INICIO → crear TDA → menú → seleccionar insertar/eliminar/mostrar → imprimir Antes → ejecutar operación validada → imprimir Después → repetir → FIN.

Menú real de ejecución por teclado:

1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir



Entrada por teclado	Acción	Resultado esperado / verificado
2, 0	Eliminar en vacía	Aviso
1, 10, 0	Insertar	[10]
1, 30, 1	Insertar final	[10 30]
1, 20, 1	Insertar medio	[10 20 30]
2, 0	Eliminar primero	[20 30]
3, 30	Eliminar valor	[20]
0	Salir	Saliendo...

16. Prueba de escritorio, resultados verificados y análisis solicitado

La siguiente prueba resume el comportamiento comprobado del TDA con una secuencia representativa de operaciones:

Paso	Operación	Antes	Después/resultado
1	Eliminar posición 0	[lista vacía]	No se puede eliminar
2	Insertar 10 en 0	[]	[10]
3	Insertar 30 en 1	[10]	[10 30]
4	Insertar 20 en 1	[10 30]	[10 20 30]
5	Eliminar posición 0	[10 20 30]	[20 30]
6	Eliminar valor 30	[20 30]	[20]

Pregunta textual del enunciado: Explique qué ocurre en los siguientes casos: lista vacía, lista con un solo nodo, lista con varios nodos.

Respuesta: Lista vacía: no se puede eliminar y la primera inserción crea un nodo que se apunta a sí mismo. Un solo nodo: al eliminarlo, ultimo pasa a null y tamaño a 0. Varios nodos: se reconectan el nodo anterior y el siguiente; si se elimina el último, también se actualiza la referencia ultimo.



Ejercicio 3. Simulación de turnos Round-Robin

1. Ejercicio

Construya un programa en Java que simule la atención de procesos usando una lista circular bajo el algoritmo Round-Robin. Cada proceso tiene nombre y tiempo restante; el quantum será de 2 unidades; si el proceso no termina, vuelve al final del ciclo; si termina, debe eliminarse de la lista. Muestre paso a paso el orden de ejecución y el estado de la lista después de cada turno.

2. Solución propuesta

La solución separa el contrato abstracto, la implementación de la estructura circular y la interacción con el usuario. Las responsabilidades se distribuyen de la siguiente manera:

Clase / módulo	Responsabilidad
ColaCircularProcesosTDA	Define agregar procesos, consultar vacío, mostrar estado y simular.
SimuladorRoundRobinImpl	Implementa la cola circular y la política Round-Robin.
Proceso	Clase interna con nombre, tiempoRestante y siguiente.
PruebaRoundRobin	Solicita quantum, agrega procesos e inicia la simulación.

3. ¿Qué es un TDA y dónde está implementado?

Un Tipo de Dato Abstracto define las operaciones disponibles y las reglas del dato sin obligar al usuario a conocer la representación interna. En este ejercicio, el contrato está en la interfaz y la implementación concreta se encuentra en la clase indicada a continuación:

```
public interface ColaCircularProcesosTDA {  
    void agregarProceso(String nombre, int tiempoRestante);  
    boolean estaVacia();  
    void mostrarEstado();  
    void simular();  
}
```

La implementación real del contrato corresponde a SimuladorRoundRobinImpl implements ColaCircularProcesosTDA. El programa principal trabaja con una referencia del tipo de la interfaz, lo que mantiene separado el uso del TDA de sus detalles internos.

4. Relación entre requisitos y código

Requisito del enunciado	Aplicación en el código Java
Proceso: nombre/tiempo	Clase interna Proceso
Quantum	campo final quantum
No termina: vuelve al final	ultimo = actual
Termina: eliminar	ultimo.siguiete = actual.siguiete
Estado por turno	mostrarEstado() dentro de simular()



5. Bibliotecas y entrada/salida

```
import java.util.Scanner;
```

La clase de prueba utiliza Scanner para capturar las opciones y datos ingresados por teclado. La salida se realiza con System.out.print y System.out.println. La estructura circular no necesita colecciones de java.util porque los enlaces se gestionan directamente mediante nodos.

6. Estructura principal del TDA

La representación interna se encuentra encapsulada en la clase de implementación. Sus campos permiten conservar el estado del TDA sin exponer directamente los nodos al programa cliente.

```
private static class Proceso {  
    String nombre;  
    int tiempoRestante;  
    Proceso siguiente;  
}  
  
private Proceso ultimo;  
private int cantidad;  
private final int quantum;
```

7. Implementación de la interfaz

La clase concreta declara SimuladorRoundRobinImpl implements ColaCircularProcesosTDA. Los métodos definidos por la interfaz se implementan con @Override, de modo que el programa cliente puede invocarlos a través del tipo abstracto sin depender de la clase concreta.

```
public class SimuladorRoundRobinImpl implements ColaCircularProcesosTDA {  
    // métodos @Override del TDA  
}
```

8. Nodo o elemento de la estructura circular

El elemento enlazado es Proceso. Además del enlace siguiente, almacena el nombre del proceso y su tiempo restante, datos necesarios para la simulación.

9. Construcción e instanciación del TDA

```
int quantum = leerEntero(sc);  
ColaCircularProcesosTDA simulador =  
    new SimuladorRoundRobinImpl(quantum);
```

La instanciación crea la implementación concreta, pero la variable del cliente se declara con el tipo de la interfaz. Esto permite utilizar el contrato del TDA y mantener ocultos los detalles de los nodos.

10. Operaciones principales



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Operación	Resultado / descripción
agregarProceso(nombre,t)	Inserta un proceso al final del ciclo.
mostrarEstado()	Muestra nombre(tiempoRestante) para una vuelta completa.
simular()	Atiende el frente, consume min(quantum, restante), elimina si termina o lo rota al final.
estaVacia()	Devuelve ultimo == null.

11. Funcionamiento interno de las operaciones

```
Proceso actual = ultimo.siguiente;  
int ejecutado = Math.min(quantum, actual.tiempoRestante);  
actual.tiempoRestante -= ejecutado;  
  
if (actual.tiempoRestante <= 0) {  
    ultimo.siguiente = actual.siguiente;  
    cantidad--;  
} else {  
    ultimo = actual;  
}
```

El frente de la cola es ultimo.siguiente. Si el proceso conserva tiempo, convertirlo en ultimo produce la rotación Round-Robin. Si termina, se desconecta del frente.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / uso
tiempoRestante, cantidad, quantum	int	Primitivo	Tiempo de CPU, número de procesos y quantum.
nombre	String	Referencia	Identificador del proceso.
ultimo, siguiente	Proceso	Referencia	Estructura circular de la cola.
simulador	ColaCircularProcesosTDA	Referencia	Acceso abstracto al simulador.



13. Diagrama de clases

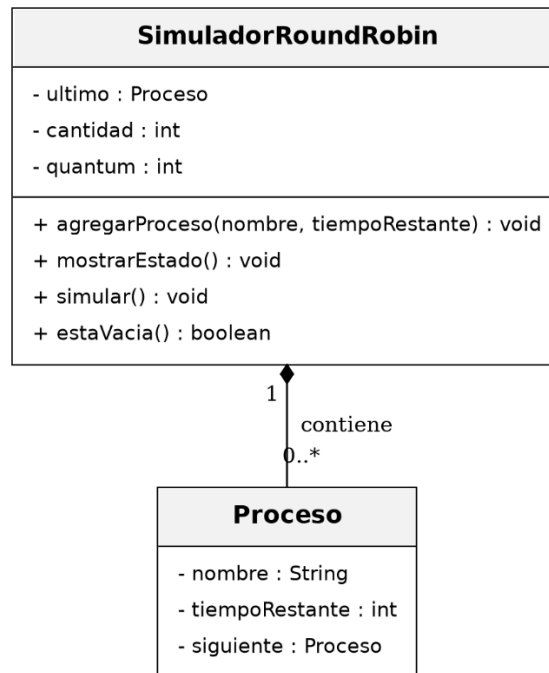


Figura 3. Diagrama de clases de SimuladorRoundRobin y Proceso.

14. Especificación del TDA

Estado e invariantes de representación:

- Si hay procesos, ultimo.siguiete es el frente.
- Todos los procesos activos forman un único ciclo.
- cantidad coincide con los procesos activos.
- Un proceso no terminado rota al final sin crear otro nodo.
- Un proceso terminado deja de pertenecer al ciclo.
- Para la práctica se usa quantum = 2.

Las operaciones públicas válidas son las definidas en la interfaz mostrada en el paso 3. Mientras se respeten los invariantes anteriores, la estructura conserva su comportamiento circular después de cada operación.

15. Diagrama de flujo y menú de ejecución

Flujo general del programa:

INICIO → leer quantum → crear simulador → agregar procesos → mostrar cola → iniciar simular → tomar frente → ejecutar hasta quantum → ¿terminó? sí: eliminar / no: rotar → mostrar estado → repetir hasta cola vacía → FIN.

Menú real de ejecución por teclado:

1. Agregar proceso
2. Mostrar estado de la cola



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



3. Iniciar simulación (hasta que todos terminen)
0. Salir

Entrada por teclado	Acción	Resultado esperado / verificado
2	Quantum	quantum=2
1, P1, 5	Agregar	P1
1, P2, 3	Agregar	P2
1, P3, 4	Agregar	P3
2	Mostrar	P1(5) P2(3) P3(4)
3	Simular	7 turnos; todos finalizan
0	Salir	Saliendo...

16. Prueba de escritorio, resultados verificados y análisis solicitado

La siguiente prueba resume el comportamiento comprobado del TDA con una secuencia representativa de operaciones:

Turno	Proceso	Antes	Ejecuta	Después	Cola resultante
Inicial	—	—	—	—	P1(5) P2(3) P3(4)
1	P1	5	2	3	P2(3) P3(4) P1(3)
2	P2	3	2	1	P3(4) P1(3) P2(1)
3	P3	4	2	2	P1(3) P2(1) P3(2)
4	P1	3	2	1	P2(1) P3(2) P1(1)
5	P2	1	1	0	P3(2) P1(1)
6	P3	2	2	0	P1(1)
7	P1	1	1	0	[vacía]

Pregunta textual del enunciado: Muestre paso a paso el orden de ejecución de los procesos y el estado de la lista después de cada turno.

Respuesta: Con P1=5, P2=3, P3=4 y quantum=2, el orden verificado es P1, P2, P3, P1, P2, P3, P1. Los procesos que conservan tiempo restante rotan al final; P2 termina en el turno 5, P3 en el 6 y P1 en el 7.



Ejercicio 4. Problema de Josephus

1. Ejercicio

Implemente en Java el problema de Josephus usando una lista circular. Dadas n personas ubicadas en círculo, elimine cada k -ésima persona hasta que quede solo una. Probar con $n = 5$ y $k = 2$, y con $n = 7$ y $k = 3$; mostrar el orden de eliminación e indicar el superviviente final. Explique por qué una lista circular es adecuada.

2. Solución propuesta

La solución separa el contrato abstracto, la implementación de la estructura circular y la interacción con el usuario. Las responsabilidades se distribuyen de la siguiente manera:

Clase / módulo	Responsabilidad
CirculoJosephusTDA	Define construir(n) y resolver(k).
CirculoJosephusImpl	Construye el círculo y elimina cada k -ésimo nodo.
Nodo	Guarda el número de persona y el enlace siguiente.
PruebaJosephus	Solicita n y k , valida y muestra el superviviente.

3. ¿Qué es un TDA y dónde está implementado?

Un Tipo de Dato Abstracto define las operaciones disponibles y las reglas del dato sin obligar al usuario a conocer la representación interna. En este ejercicio, el contrato está en la interfaz y la implementación concreta se encuentra en la clase indicada a continuación:

```
public interface CirculoJosephusTDA {  
    void construir(int n);  
    int resolver(int k);  
}
```

La implementación real del contrato corresponde a `CirculoJosephusImpl` implements `CirculoJosephusTDA`. El programa principal trabaja con una referencia del tipo de la interfaz, lo que mantiene separado el uso del TDA de sus detalles internos.

4. Relación entre requisitos y código

Requisito del enunciado	Aplicación en el código Java
Construir n personas	<code>construir(int n)</code>
Eliminar cada k -ésima	<code>resolver(int k)</code>
Orden de eliminación	<code>System.out.print(actual.valor)</code>
Superviviente	<code>return actual.valor</code>

5. Bibliotecas y entrada/salida

```
import java.util.Scanner;
```




La clase de prueba utiliza Scanner para capturar las opciones y datos ingresados por teclado. La salida se realiza con System.out.print y System.out.println. La estructura circular no necesita colecciones de java.util porque los enlaces se gestionan directamente mediante nodos.

6. Estructura principal del TDA

La representación interna se encuentra encapsulada en la clase de implementación. Sus campos permiten conservar el estado del TDA sin exponer directamente los nodos al programa cliente.

```
private static class Nodo {  
    int valor;  
    Nodo siguiente;  
    Nodo(int valor) { this.valor = valor; }  
}  
  
private Nodo ultimo;
```

7. Implementación de la interfaz

La clase concreta declara CirculoJosephusImpl implements CirculoJosephusTDA. Los métodos definidos por la interfaz se implementan con @Override, de modo que el programa cliente puede invocarlos a través del tipo abstracto sin depender de la clase concreta.

```
public class CirculoJosephusImpl implements CirculoJosephusTDA {  
    // métodos @Override del TDA  
}
```

8. Nodo o elemento de la estructura circular

La clase interna Nodo es un detalle de implementación. Cada nodo contiene el dato del ejercicio y la referencia siguiente. La circularidad se obtiene evitando que el último enlace sea null y conectándolo con el primer nodo.

9. Construcción e instanciación del TDA

```
CirculoJosephusTDA josephus = new CirculoJosephusImpl();  
josephus.construir(n);  
int sobreviviente = josephus.resolver(k);
```

La instanciación crea la implementación concreta, pero la variable del cliente se declara con el tipo de la interfaz. Esto permite utilizar el contrato del TDA y mantener ocultos los detalles de los nodos.

10. Operaciones principales

Operación	Resultado / descripción
construir(n)	Crea personas numeradas 1..n e inserta cada una al final del círculo.
resolver(k)	Avanza k-1 enlaces, elimina la persona actual y continúa desde la siguiente hasta dejar una.



11. Funcionamiento interno de las operaciones

```
Nodo actual = ultimo.siguiete;  
Nodo anterior = ultimo;  
  
while (actual.siguiete != actual) {  
    for (int i = 1; i < k; i++) {  
        anterior = actual;  
        actual = actual.siguiete;  
    }  
    anterior.siguiete = actual.siguiete;  
    if (actual == ultimo) ultimo = anterior;  
    actual = actual.siguiete;  
}
```

El conteo avanza k-1 veces para que actual quede sobre la persona que debe eliminarse. La eliminación se realiza haciendo que anterior salte al siguiente de actual.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / uso
n, k, valor, opcion	int	Primitivo	Cantidad de personas, paso y número de persona.
ultimo, actual, anterior	Nodo	Referencia	Recorrido y eliminación dentro del círculo.
josephus	CirculoJosephusTDA	Referencia	Contrato abstracto del problema.

13. Diagrama de clases

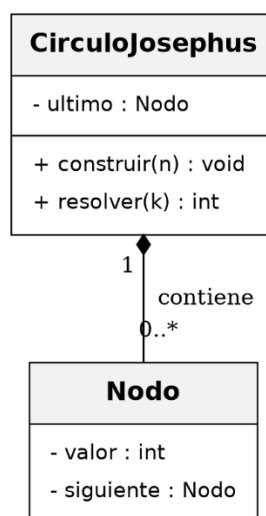


Figura 4. Diagrama de clases de CirculoJosephus y Nodo.



14. Especificación del TDA

Estado e invariantes de representación:

- Después de construir, las personas 1..n forman un único ciclo.
- ultimo.siguiete apunta a la primera persona activa.
- Cada eliminación reduce en uno el conjunto activo sin romper el ciclo.
- Si se elimina ultimo, se actualiza a anterior.
- Con una persona restante, el nodo se apunta a sí mismo.

Las operaciones públicas válidas son las definidas en la interfaz mostrada en el paso 3. Mientras se respeten los invariantes anteriores, la estructura conserva su comportamiento circular después de cada operación.

15. Diagrama de flujo y menú de ejecución

Flujo general del programa:

INICIO → leer n y k → validar > 0 → construir círculo 1..n → ubicar actual en 1 → contar k → eliminar → continuar desde siguiente → ¿queda uno? no: repetir / sí: mostrar superviviente → FIN.

Menú real de ejecución por teclado:

1. Resolver (ingresar n y k)
0. Salir

Entrada por teclado	Acción	Resultado esperado / verificado
1, 5, 2	Primer caso	Elimina 2,4,1,5; sobrevive 3
1, 7, 3	Segundo caso	Elimina 3,6,2,7,5,1; sobrevive 4
0	Salir	Saliendo...

16. Prueba de escritorio, resultados verificados y análisis solicitado

La siguiente prueba resume el comportamiento comprobado del TDA con una secuencia representativa de operaciones:

Caso	Orden de eliminación verificado	Superviviente
n=5, k=2	2, 4, 1, 5	3
n=7, k=3	3, 6, 2, 7, 5, 1	4

Pregunta textual del enunciado: ¿Por qué una lista circular es una estructura adecuada para este problema?

Respuesta: Porque el conteo de Josephus debe continuar desde el inicio al superar la última persona. En una lista circular, el último nodo enlaza directamente con el primero, por lo que el recorrido cíclico ocurre de forma natural y la eliminación se realiza reconectando enlaces.



Ejercicio 5. Playlist musical circular

1. Ejercicio

Diseñe un programa en Java que represente una playlist musical circular. El sistema debe permitir agregar canciones al inicio y al final, mostrar la playlist completa, reproducir la siguiente canción, eliminar una canción por nombre y volver automáticamente a la primera canción al llegar al final. Explique la ventaja frente a una lista lineal.

2. Solución propuesta

La solución separa el contrato abstracto, la implementación de la estructura circular y la interacción con el usuario. Las responsabilidades se distribuyen de la siguiente manera:

Clase / módulo	Responsabilidad
PlaylistCircularTDA	Declara las operaciones públicas de la playlist.
PlaylistCircularImpl	Gestiona canciones, ultimo, actual y cantidad.
Cancion	Clase interna con nombre y siguiente.
PruebaPlaylist	Presenta el menú y captura nombres por teclado.

3. ¿Qué es un TDA y dónde está implementado?

Un Tipo de Dato Abstracto define las operaciones disponibles y las reglas del dato sin obligar al usuario a conocer la representación interna. En este ejercicio, el contrato está en la interfaz y la implementación concreta se encuentra en la clase indicada a continuación:

```
public interface PlaylistCircularTDA {  
    void agregarAlInicio(String nombre);  
    void agregarAlFinal(String nombre);  
    void mostrarPlaylist();  
    void reproducirSiguiente();  
    boolean eliminarPorNombre(String nombre);  
    boolean estaVacía();  
}
```

La implementación real del contrato corresponde a PlaylistCircularImpl implements PlaylistCircularTDA. El programa principal trabaja con una referencia del tipo de la interfaz, lo que mantiene separado el uso del TDA de sus detalles internos.

4. Relación entre requisitos y código

Requisito del enunciado	Aplicación en el código Java
Agregar inicio	agregarAlInicio(String)
Agregar final	agregarAlFinal(String)
Mostrar	mostrarPlaylist()
Siguiente	reproducirSiguiente()
Eliminar por nombre	eliminarPorNombre(String)
Volver al primero	enlace circular ultimo.siguiente



5. Bibliotecas y entrada/salida

```
import java.util.Scanner;
```

La clase de prueba utiliza Scanner para capturar las opciones y datos ingresados por teclado. La salida se realiza con System.out.print y System.out.println. La estructura circular no necesita colecciones de java.util porque los enlaces se gestionan directamente mediante nodos.

6. Estructura principal del TDA

La representación interna se encuentra encapsulada en la clase de implementación. Sus campos permiten conservar el estado del TDA sin exponer directamente los nodos al programa cliente.

```
private static class Cancion {  
    String nombre;  
    Cancion siguiente;  
    Cancion(String nombre) { this.nombre = nombre; }  
}  
  
private Cancion ultimo;  
private Cancion actual;  
private int cantidad;
```

7. Implementación de la interfaz

La clase concreta declara PlaylistCircularImpl implements PlaylistCircularTDA. Los métodos definidos por la interfaz se implementan con @Override, de modo que el programa cliente puede invocarlos a través del tipo abstracto sin depender de la clase concreta.

```
public class PlaylistCircularImpl implements PlaylistCircularTDA {  
    // métodos @Override del TDA  
}
```

8. Nodo o elemento de la estructura circular

El elemento enlazado es Cancion. Cada objeto conserva el nombre y la referencia siguiente. La referencia siguiente del último elemento conduce a la primera canción.

9. Construcción e instanciación del TDA

```
PlaylistCircularTDA playlist = new PlaylistCircularImpl();
```

La instanciación crea la implementación concreta, pero la variable del cliente se declara con el tipo de la interfaz. Esto permite utilizar el contrato del TDA y mantener ocultos los detalles de los nodos.

10. Operaciones principales



Operación	Resultado / descripción
agregarAlInicio(nombre)	Inserta como primera canción; en la primera inserción también inicializa actual.
agregarAlFinal(nombre)	Inserta después del último y actualiza ultimo.
mostrarPlaylist()	Recorre una vuelta y marca actual entre corchetes.
reproducirSiguiente()	Ejecuta actual = actual.siguiente; desde la última vuelve a la primera.
eliminarPorNombre(nombre)	Elimina la primera coincidencia y actualiza ultimo/actual cuando corresponde.
estaVacia()	Devuelve ultimo == null.

11. Funcionamiento interno de las operaciones

actual = actual.siguiente;

```
if (nodo.nombre.equals(nombre)) {  
    anterior.siguiente = nodo.siguiente;  
    if (nodo == ultimo) ultimo = anterior;  
    if (nodo == actual) actual = nodo.siguiente;  
    cantidad--;  
}
```

La canción activa avanza únicamente siguiendo el enlace siguiente. Por eso, cuando actual se encuentra en la última canción, el siguiente paso llega automáticamente a la primera.

12. Tipos de datos utilizados

Variables	Tipo	Categoría	Descripción / uso
nombre	String	Referencia	Nombre de la canción.
cantidad, opcion	int	Primitivo	Número de canciones y control del menú.
ultimo, actual, siguiente	Cancion	Referencia	Orden circular y canción activa.
playlist	PlaylistCircularTDA	Referencia	Contrato del TDA.



13. Diagrama de clases

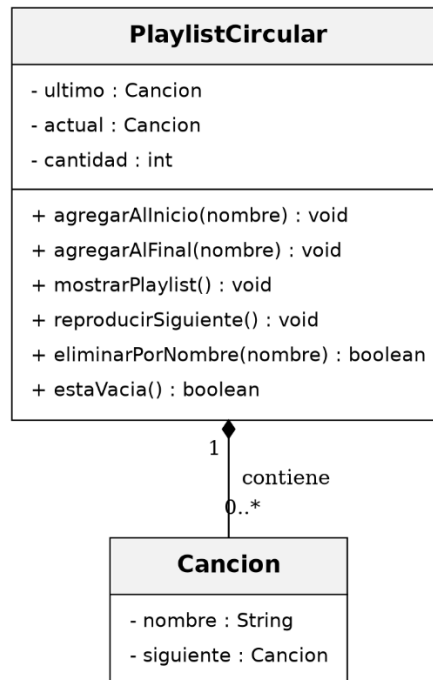


Figura 5. Diagrama de clases de PlaylistCircular y Cancion.

14. Especificación del TDA

Estado e invariantes de representación:

- Lista vacía: ultimo == null, actual == null y cantidad == 0.
- Lista no vacía: ultimo.siguiente es la primera canción.
- actual pertenece al ciclo mientras haya canciones.
- Seguir siguiente produce un recorrido circular sin null.
- cantidad coincide con el número de canciones.
- Si se elimina actual, se selecciona una canción válida mientras queden elementos.

Las operaciones públicas válidas son las definidas en la interfaz mostrada en el paso 3. Mientras se respeten los invariantes anteriores, la estructura conserva su comportamiento circular después de cada operación.

15. Diagrama de flujo y menú de ejecución

Flujo general del programa:

INICIO → crear playlist → menú → agregar/mostrar/siguiente/eliminar → si se reproduce siguiente: actual = actual.siguiente → desde último se retorna al primero → repetir → FIN.

Menú real de ejecución por teclado:

1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



- 4. Reproducir siguiente canción
- 5. Eliminar canción por nombre
- 0. Salir

Entrada por teclado	Acción	Resultado esperado / verificado
1, A	Agregar inicio	[A]
2, B	Agregar final	[A] -> B
2, C	Agregar final	[A] -> B -> C
4	Siguiente	B
4	Siguiente	C
4	Siguiente	A
5, B	Eliminar	B eliminada
0	Salir	Saliendo...

16. Prueba de escritorio, resultados verificados y análisis solicitado

La siguiente prueba resume el comportamiento comprobado del TDA con una secuencia representativa de operaciones:

Paso	Operación	Resultado verificado
1	Agregar A al inicio	Playlist: [A]
2	Agregar B al final	[A] -> B
3	Agregar C al final	[A] -> B -> C
4	Siguiente	B
5	Siguiente	C
6	Siguiente	A; retorno automático
7	Eliminar B	Canción "B" eliminada

Pregunta textual del enunciado: Explique qué ventaja tiene usar una lista circular en este caso frente a una lista lineal.

Respuesta: La reproducción continua se obtiene directamente de la estructura. La última canción apunta a la primera, por lo que avanzar desde el final no requiere reiniciar manualmente el índice o buscar nuevamente el comienzo; además, las inserciones y eliminaciones se resuelven modificando enlaces.



Figura 6. Ejecución del Primer Ejercicio.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
=====
LISTA CIRCULAR - INSERCIÓN Y ELIMINACIÓN CONTROLADA
=====
1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir
Seleccione una opción: 1
Valor a insertar: 10
Posición (0 = inicio, 0 = final): 0
Antes: [ lista vacía ]
Después: [ 10 ]

=====
LISTA CIRCULAR - INSERCIÓN Y ELIMINACIÓN CONTROLADA
=====
1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir
Seleccione una opción: 1
Valor a insertar: 20
Posición (0 = inicio, 1 = final): 1
Antes: [ 10 ]
Después: [ 10 20 ]
```

```
=====
LISTA CIRCULAR - INSERCIÓN Y ELIMINACIÓN CONTROLADA
=====
1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir
Seleccione una opción: 2
Posición a eliminar: 0
Antes: [ 10 20 ]
Después: [ 20 ]

=====
LISTA CIRCULAR - INSERCIÓN Y ELIMINACIÓN CONTROLADA
=====
1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir
Seleccione una opción: 4
[ 20 ]

=====
LISTA CIRCULAR - INSERCIÓN Y ELIMINACIÓN CONTROLADA
=====
1. Insertar en una posición específica
2. Eliminar por posición
3. Eliminar por valor
4. Mostrar la lista
0. Salir
Seleccione una opción: 0
Saliendo...
```

Figura 7. Ejecución del Segundo Ejercicio.

```
Ingrese el valor del quantum: 2

=====
SIMULACIÓN ROUND-ROBIN (quantum = 2)
=====
1. Agregar proceso
2. Mostrar estado de la cola
3. Iniciar simulación (hasta que todos terminen)
0. Salir
Seleccione una opción: 1
Nombre del proceso: P1
Tiempo restante: 3
Proceso agregado.

=====
SIMULACIÓN ROUND-ROBIN (quantum = 2)
=====
1. Agregar proceso
2. Mostrar estado de la cola
3. Iniciar simulación (hasta que todos terminen)
0. Salir
Seleccione una opción: 1
Nombre del proceso: P2
Tiempo restante: 2
Proceso agregado.

=====
SIMULACIÓN ROUND-ROBIN (quantum = 2)
=====
1. Agregar proceso
2. Mostrar estado de la cola
3. Iniciar simulación (hasta que todos terminen)
0. Salir
Seleccione una opción: 3
Estado inicial:
Cola de procesos: [ P1(3) P2(2) ]

Turno 1: ejecutando P1 (tiempo restante antes: 3)
Se ejecuta durante 2 unidades (quantum=2).
P1 no ha terminado (restante: 1), vuelve al final del ciclo.
Cola de procesos: [ P2(2) P1(1) ]

Turno 2: ejecutando P2 (tiempo restante antes: 2)
Se ejecuta durante 2 unidades (quantum=2).
P2 ha terminado y se elimina de la lista.
Cola de procesos: [ P1(1) ]

Turno 3: ejecutando P1 (tiempo restante antes: 1)
Se ejecuta durante 1 unidades (quantum=2).
P1 ha terminado y se elimina de la lista.
Cola de procesos: [ vacía ]

Todos los procesos han finalizado.

=====
SIMULACIÓN ROUND-ROBIN (quantum = 2)
=====
1. Agregar proceso
2. Mostrar estado de la cola
3. Iniciar simulación (hasta que todos terminen)
0. Salir
Seleccione una opción: 0
Saliendo...
PS C:\Users\luiss\Downloads\Java\Ejercicio3_RoundRobin>
```

Figura 8. Ejecución del Tercero Ejercicio.



```
=====
PROBLEMA DE JOSEPHUS
=====
1. Resolver (ingresar n y k)
0. Salir
Seleccione una opción: 1
Número de personas (n): 5
Paso de conteo (k): 2
Orden de eliminación: 2 4 1 5
Sobreviviente: 3

=====
PROBLEMA DE JOSEPHUS
=====
1. Resolver (ingresar n y k)
0. Salir
Seleccione una opción: 0
Saliendo...
PS C:\Users\luiss\Downloads\Java\Ejercicio4_Josephus> |
```

Figura 9. Ejecución del Cuarto Ejercicio.

```
=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 2
Nombre de la canción: A
Canción agregada al final.

=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 2
Nombre de la canción: B
Canción agregada al final.

=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 3
Playlist: [A] -> B -> (vuelve a: A)

=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 4
Reproduciendo ahora: B

=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 4
Reproduciendo ahora: A

=====
PLAYLIST MUSICAL CIRCULAR
=====
1. Agregar canción al inicio
2. Agregar canción al final
3. Mostrar playlist completa
4. Reproducir siguiente canción
5. Eliminar canción por nombre
0. Salir
Seleccione una opción: 0
Saliendo...
PS C:\Users\luiss\Downloads\Java\Ejercicio5_PlaylistCircular> |
```

Figura 10. Ejecución del Quinto Ejercicio.

A continuación, adjunto el enlace del repositorio donde se podrá visualizar los códigos en Java correspondientes a los 5 ejercicios desarrollados (Lista Circular Básica, Inserción y Eliminación Controlada, Simulación Round-Robin, Problema de Josephus y Playlist Circular), además de las capturas de ejecución de cada programa y el presente documento en formato .pdf. Los diagramas de clases (UML) de cada ejercicio ya se encuentran incorporados dentro del documento, en la sección correspondiente a cada uno.

https://github.com/luuissilva/Estructuras_de_Datos_Tara03_Listas_Luis_Silva.git